```python
"""1. Descriptive Statistics & Bar Chart
Marks obtained by 12 students: [45, 56, 67, 72, 61, 58, 49, 70, 64, 55, 60, 68].
Tasks:
• Compute Mean, Median, Mode, Standard Deviation using Python
• Draw a bar chart
• Comment on performance   """

##########################################################################################


import numpy as np
import matplotlib.pyplot as plt
from statistics import mode
marks = [45, 56, 67, 72, 61, 58, 49, 70, 64, 55, 60, 68]
# Descriptive Statistics
mean = np.mean(marks)
median = np.median(marks)
mode_val = mode(marks)
std_dev = np.std(marks)
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode_val)
print("Standard Deviation:", std_dev)
# Performance Comment
print("\nPerformance Comment:")
if mean >= 60:
    print("Overall performance is good and above average.")
else:
    print("Overall performance is average.")
if std_dev < 10:
    print("Marks are consistent with low variation.")
else:
    print("Marks show high variation.")
# Bar Chart
plt.bar(range(1, 13), marks)
plt.xlabel("Student Number")
plt.ylabel("Marks")
plt.title("Marks of 12 Students")
plt.show()



##########################################################################################
```

```
##########################################################################################
"""2. Monthly Sales Analysis using Line Chart
Sales (in thousands): Jan-120, Feb-135, Mar-150, Apr-145, May-160, Jun-170.
Tasks:
• Compute Mean and Range
• Plot line chart
• Identify highest and lowest sales months """

##########################################################################################

import numpy as np
import matplotlib.pyplot as plt
# Data
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 135, 150, 145, 160, 170]
# Mean and Range
mean_sales = np.mean(sales)
range_sales = max(sales) - min(sales)
print("Mean Sales:", mean_sales)
print("Range of Sales:", range_sales)
# Highest and Lowest Month
highest_month = months[sales.index(max(sales))]
lowest_month = months[sales.index(min(sales))]
print("Highest Sales Month:", highest_month)
print("Lowest Sales Month:", lowest_month)
# Line Chart
plt.plot(months, sales, marker='o')
plt.xlabel("Months")
plt.ylabel("Sales (in thousands)")
plt.title("Monthly Sales Analysis")
plt.show()


##########################################################################################


"""3. Pie Chart for Course Distribution
Data Science-40, AI-30, Cyber Security-20, Cloud Computing-10.
Tasks:
• Plot pie chart
• Calculate percentage distribution
• Interpret results"""

##########################################################################################

import matplotlib.pyplot as plt
# Data
courses = ["Data Science", "AI", "Cyber Security", "Cloud Computing"]
values = [40, 30, 20, 10]
# Total
total = sum(values)
# Percentage Distribution
percentages = [(v / total) * 100 for v in values]
print("Percentage Distribution:")
for i in range(len(courses)):
    print(f"{courses[i]}: {percentages[i]:.2f}%")
# Pie Chart
plt.pie(values, labels=courses, autopct='%1.1f%%')
```

```
plt.title("Course Distribution")
plt.show()
# Interpretation
print("\nInterpretation:")
print("Data Science has the highest enrollment (40%).")
print("Cloud Computing has the lowest enrollment (10%).")
print("Majority of students prefer Data Science and AI.")



##########################################################################################


"""1. Temperature Data Analysis
Temperatures: [28, 30, 32, 31, 29, 33, 34, 35, 36, 34, 33, 32, 31, 30, 29].
Tasks:
• Compute Mean, Median, Variance, Standard Deviation, Quartiles.
• Plot line chart and histogram
• Analyze trend """

##########################################################################################

import numpy as np
import matplotlib.pyplot as plt
# Data
temps = [28, 30, 32, 31, 29, 33, 34, 35, 36, 34, 33, 32, 31, 30, 29]
# Statistical Measures
mean = np.mean(temps)
median = np.median(temps)
variance = np.var(temps)
std_dev = np.std(temps)
quartiles = np.percentile(temps, [25, 50, 75])
print("Mean:", mean)
print("Median:", median)
print("Variance:", variance)
print("Standard Deviation:", std_dev)
print("Quartiles (Q1, Q2, Q3):", quartiles)
# Line Chart
plt.figure()
plt.plot(temps, marker='o')
plt.title("Temperature Trend")
plt.xlabel("Day")
plt.ylabel("Temperature")
plt.show()
# Histogram
plt.figure()
plt.hist(temps, bins=5)
plt.title("Temperature Distribution")
plt.xlabel("Temperature")
plt.ylabel("Frequency")
plt.show()
# Trend Analysis
print("\nTrend Analysis:")
if temps[-1] > temps[0]:
    print("Overall increasing temperature trend.")
else:
    print("No overall increasing trend.")
```

```python
"""2. Weekly Expense Analysis
Expenses : Mon-250, Tue-300, Wed-280, Thu-350, Fri-400, Sat-450, Sun-320.
Tasks:
• Compute Mean, Median, Standard Deviation, Coefficient of Variation
• Draw bar chart and pie chart
• Write inference   """

##########################################################################################

import numpy as np
import matplotlib.pyplot as plt
# Data
days = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]
expenses = [250, 300, 280, 350, 400, 450, 320]
# Calculations
mean = np.mean(expenses)
median = np.median(expenses)
std_dev = np.std(expenses)
cv = (std_dev / mean) * 100   # Coefficient of Variation %
print("Mean:", mean)
print("Median:", median)
print("Standard Deviation:", std_dev)
print("Coefficient of Variation (%):", cv)
# Bar Chart
plt.figure()
plt.bar(days, expenses)
plt.title("Weekly Expenses - Bar Chart")
plt.xlabel("Days")
plt.ylabel("Expenses")
plt.show()
# Pie Chart
plt.figure()
plt.pie(expenses, labels=days, autopct='%1.1f%%')
plt.title("Weekly Expenses Distribution")
plt.show()
# Inference
print("\nInference:")
print("Highest expense on Saturday and lowest on Monday.")
print("Expenses increase towards weekend.")
print("Moderate variation observed in weekly spending.")



##########################################################################################

"""Q.1 Central Tendency Focus
The daily revenue (₹ in thousands) of a café for 15 days is:
45, 50, 55, 60, 62, 65, 70, 72, 75, 80, 85, 90, 95, 120, 200
Using Python: 1.Compute Mean, Median and Mode.
Compare mean and median and comment on data symmetry.
Identify whether mean is a reliable measure for this dataset."""
##########################################################################################

import numpy as np
from statistics import mode
# Data (₹ in thousands)
revenue = [45, 50, 55, 60, 62, 65, 70, 72, 75, 80, 85, 90, 95, 120, 200]
```

```python
# Mean, Median, Mode
mean = np.mean(revenue)
median = np.median(revenue)
# Mode (handle no repeated values)
try:
    mode_val = mode(revenue)
except:
    mode_val = "No mode"
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode_val)
# Comment on symmetry
if mean > median:
    print("\nData is positively skewed (right-skewed).")
elif mean < median:
    print("\nData is negatively skewed (left-skewed).")
else:
    print("\nData is symmetric.")
# Reliability of mean
print("\nMean may not be reliable due to extreme value (200) affecting average.")



##########################################################################################

"""Measures of Dispersion
The number of calls received by a call center in 20 days is:
120, 135, 140, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 210, 230, 260, 300, 350, 420
Using Python:
Compute Range, Variance and Standard Deviation.
Compute Coefficient of Variation.
Comment on variability and stability of call volume."""
import numpy as np
# Data
calls = [120, 135, 140, 150, 155, 160, 165, 170, 175, 180,
         185, 190, 195, 200, 210, 230, 260, 300, 350, 420]
# Measures of Dispersion
range_val = max(calls) - min(calls)
variance = np.var(calls)
std_dev = np.std(calls)
# Coefficient of Variation
mean = np.mean(calls)
cv = (std_dev / mean) * 100
print("Range:", range_val)
print("Variance:", variance)
print("Standard Deviation:", std_dev)
print("Coefficient of Variation (%):", cv)
# Comment on variability
print("\nComment:")
if cv < 20:
    print("Low variability and stable call volume.")
elif cv < 40:
    print("Moderate variability in call volume.")
else:
    print("High variability and less stability in call volume.")
```

```
##########################################################################################
"""Quartiles and IQR
Monthly expenditure (₹ in thousands) of 25 families:
10, 12, 14, 15, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 45, 50, 55, 60, 70, 85, 100, 150
Using Python:
Compute Q1, Q2, Q3
Find Interquartile Range (IQR)
Identify outliers using IQR method
Interpret spending inequality"""

##########################################################################################

import numpy as np
# Data
expenditure = [10, 12, 14, 15, 16, 18, 20, 22, 24, 26, 28, 30,
               32, 34, 36, 38, 40, 45, 50, 55, 60, 70, 85, 100, 150]
# Quartiles
Q1 = np.percentile(expenditure, 25)
Q2 = np.percentile(expenditure, 50)   # Median
Q3 = np.percentile(expenditure, 75)
# IQR
IQR = Q3 - Q1
# Outlier Limits
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Outliers
outliers = [x for x in expenditure if x < lower_bound or x > upper_bound]
print("Q1:", Q1)
print("Q2 (Median):", Q2)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Outliers:", outliers)
# Interpretation
print("\nInterpretation:")
if outliers:
    print("There is spending inequality due to high outlier values.")
else:
    print("Spending distribution is fairly balanced.")




##########################################################################################


"""Skewness
The test scores of 22 students are:
32, 35, 38, 40, 42, 45, 48, 50, 52, 55, 58, 60, 62, 65, 68, 70, 72, 75, 80, 85, 90, 98
Using Python:
Compute Skewness
Plot Histogram (optional in lab)
Determine whether the distribution is symmetric, left-skewed or right-skewed
Justify numerically"""

##########################################################################################
```

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skew
# Data

scores = [32, 35, 38, 40, 42, 45, 48, 50, 52, 55, 58,
          60, 62, 65, 68, 70, 72, 75, 80, 85, 90, 98]
# Skewness
skewness = skew(scores)
print("Skewness:", skewness)
# Histogram
plt.hist(scores, bins=6)
plt.title("Histogram of Test Scores")
plt.xlabel("Scores")
plt.ylabel("Frequency")
plt.show()
# Interpretation
if skewness > 0:
    print("\nDistribution is Right-Skewed (Positively Skewed).")
elif skewness < 0:
    print("\nDistribution is Left-Skewed (Negatively Skewed).")
else:
    print("\nDistribution is Symmetric.")
print("Numerical Justification: Skewness value =", skewness)

##########################################################################################

"""Kurtosis
Daily stock price changes (%) over 25 days:
-6, -4, -3, -2, -1, -1, 0, 0, 1, 1, 2, 2, 3, 3, 4, 5, 6, 7, 8, 10, 12, 15, 18, 25, 40
Using Python:
Compute Kurtosis
Identify whether the data is leptokurtic, mesokurtic or platykurtic
Explain what this indicates about market risk"""
import numpy as np
from scipy.stats import kurtosis
# Data
changes = [-6, -4, -3, -2, -1, -1, 0, 0, 1, 1, 2, 2, 3, 3,
           4, 5, 6, 7, 8, 10, 12, 15, 18, 25, 40]
# Kurtosis (Fisher's definition: normal = 0)
kurt = kurtosis(changes)
print("Kurtosis:", kurt)
# Interpretation
if kurt > 0:
    print("\nData is Leptokurtic (heavy tails, high peak).")
elif kurt < 0:
    print("\nData is Platykurtic (light tails, flat distribution).")
else:
    print("\nData is Mesokurtic (normal distribution).")
print("\nMarket Risk Interpretation:")
if kurt > 0:
    print("Higher risk due to extreme price changes (heavy tails).")
else:
    print("Lower probability of extreme fluctuations.")

##########################################################################################
```

```python
"""Q.1 Correlation Analysis
Download the dataset advertising.csv and load it using Pandas.
1.  Compute the Pearson correlation between TV advertising budget and Sales
2.  Compute the Pearson correlation between Radio advertising budget and Sales
3.  Identify which advertising medium shows the strongest relationship with Sales
4.  Interpret the result
"""

##########################################################################################

import pandas as pd
# Load dataset (make sure advertising.csv is in same folder)
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (10)\Advertising.csv")
# Display first few rows
print(df.head())
# 1. Pearson correlation between TV and Sales
tv_corr = df['TV'].corr(df['Sales'])
# 2. Pearson correlation between Radio and Sales
radio_corr = df['Radio'].corr(df['Sales'])
print("\nPearson Correlation (TV vs Sales):", tv_corr)
print("Pearson Correlation (Radio vs Sales):", radio_corr)
# 3. Identify strongest relationship
if abs(tv_corr) > abs(radio_corr):
    print("\nTV advertising shows the strongest relationship with Sales.")
else:
    print("\nRadio advertising shows the strongest relationship with Sales.")
# 4. Interpretation
print("\nInterpretation:")
print("Higher positive correlation indicates stronger linear relationship.")
print("The advertising medium with higher correlation influences Sales more.")

##########################################################################################
"""Q.2 Correlation Matrix & Heatmap
Download the dataset advertising.csv.
1.  Generate the correlation matrix for all variables
2.  Visualize the correlation matrix using a heatmap
3.  Comment on the relationships observed
"""
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (10)\Advertising.csv")
# 1. Correlation Matrix
corr_matrix = df.corr()
print("Correlation Matrix:\n")
print(corr_matrix)
# 2. Heatmap Visualization
plt.figure(figsize=(6,5))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix Heatmap")
plt.show()
# 3. Comment on relationships
print("\nComment:")
print("TV shows strong positive correlation with Sales.")
print("Radio shows moderate positive correlation with Sales.")
print("Newspaper shows weak correlation with Sales.")
```

```python
##########################################################################################
"""Q.3 Simple Linear Regression
Download the dataset advertising.csv.
1.  Use TV as the independent variable and Sales as the dependent variable
2.  Build a simple linear regression model using sklearn
3.  Display the regression equation
4.  Predict Sales when TV advertising cost is 150
"""

##########################################################################################

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (10)\Advertising.csv")
# Independent and Dependent variables
X = df[['TV']].values      # Convert to NumPy array
y = df['Sales'].values
# Build model
model = LinearRegression()
model.fit(X, y)
# Regression equation
intercept = model.intercept_
slope = model.coef_[0]
print("Regression Equation:")
print(f"Sales = {intercept:.3f} + {slope:.3f} * TV")
# Predict when TV = 150
prediction = model.predict(np.array([[150]]))  # Proper 2D array
print("Predicted Sales for TV = 150:", prediction[0])

##########################################################################################
"""Q.1 Multiple Correlation Analysis
Download the dataset housing.csv.
1.  Compute the correlation between RM and MEDV
2.  Compute the correlation between LSTAT and MEDV
3.  Identify positive and negative correlations
4.  Explain the significance of these relationships
"""

##########################################################################################

import pandas as pd
# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (11)\HousingData.csv")
# 1. Correlation between RM and MEDV
rm_corr = df['RM'].corr(df['MEDV'])
# 2. Correlation between LSTAT and MEDV
lstat_corr = df['LSTAT'].corr(df['MEDV'])
print("Correlation (RM vs MEDV):", rm_corr)
print("Correlation (LSTAT vs MEDV):", lstat_corr)
# 3. Identify positive and negative correlations
print("\nRelationship Type:")
if rm_corr > 0:
    print("RM and MEDV have Positive Correlation")
else:
    print("RM and MEDV have Negative Correlation")
if lstat_corr > 0:
```

```python
    print("LSTAT and MEDV have Positive Correlation")
else:
    print("LSTAT and MEDV have Negative Correlation")
# 4. Explanation
print("\nExplanation:")
print("Higher RM (rooms) generally increases house price (MEDV).")
print("Higher LSTAT (lower status population %) generally decreases house price.")

##########################################################################################


"""Q.2 Multiple Linear Regression
Download the dataset housing.csv.
1.  Build a multiple linear regression model using RM, LSTAT, and PTRATIO
2.  Display regression coefficients
3.  Predict house price for a given input
4.  Identify the most influential feature

"""
##########################################################################################

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (11)\HousingData.csv")
# Remove missing values
df = df[['RM', 'LSTAT', 'PTRATIO', 'MEDV']].dropna()
# Independent and Dependent variables
X = df[['RM', 'LSTAT', 'PTRATIO']].values
y = df['MEDV'].values
# Build model
model = LinearRegression()
model.fit(X, y)
# Coefficients
print("Intercept:", model.intercept_)
print("RM:", model.coef_[0])
print("LSTAT:", model.coef_[1])
print("PTRATIO:", model.coef_[2])
# Prediction
input_data = np.array([[6, 12, 18]])
prediction = model.predict(input_data)
print("\nPredicted House Price:", prediction[0])
# Most influential feature
features = ['RM', 'LSTAT', 'PTRATIO']
most_influential = features[np.argmax(np.abs(model.coef_))]
print("Most Influential Feature:", most_influential)
```

```
""".1 Simple Probability
Using the given Spam Email dataset:
1. Load the dataset into a pandas DataFrame.
2. Compute:
                    (a) The probability that a randomly selected email is Spam
                    (b) The probability that a randomly selected email is Ham
3. Verify that the sum of the probabilities equals 1.
4. Interpret the results. """
##########################################################################################

import pandas as pd
# Load dataset with correct encoding
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (9)\spam.csv", encoding='latin-1')
# Check column names
print(df.columns)
# Use correct target column (usually 'v1')
print("\nClass Counts:")
print(df['v1'].value_counts())
# Total emails
total_emails = len(df)
# Probability calculations
spam_count = (df['v1'] == 'spam').sum()
ham_count = (df['v1'] == 'ham').sum()
prob_spam = spam_count / total_emails
prob_ham = ham_count / total_emails
print("\nProbability of Spam:", prob_spam)
print("Probability of Ham:", prob_ham)
# Verify sum
print("\nSum of Probabilities:", prob_spam + prob_ham)
# Interpretation
print("\nInterpretation:")
if prob_spam > prob_ham:
    print("Most emails are Spam.")
else:
    print("Most emails are Ham.")


##########################################################################################


"""
Q.2 Conditional Probability
Consider the word "free" as a feature in the dataset.
1. Compute the following probabilities:
o P(Word = “free” occurs|Spam)
o P(Word = “free” occurs|Ham)
2. Compare the two probabilities.
3. Comment on whether the word “free” is a strong indicator of spam.
"""
import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (9)\spam.csv", encoding='latin-1')

# Rename columns for clarity (optional but recommended)
df = df.rename(columns={'v1': 'Category', 'v2': 'Message'})

# Convert messages to lowercase for proper matching
```

```python
df['Message'] = df['Message'].str.lower()

# Total spam and ham emails
total_spam = (df['Category'] == 'spam').sum()
total_ham = (df['Category'] == 'ham').sum()

# Emails containing the word "free"
spam_free = df[(df['Category'] == 'spam') & (df['Message'].str.contains("free"))]
ham_free = df[(df['Category'] == 'ham') & (df['Message'].str.contains("free"))]

# 1. Conditional Probabilities
P_free_given_spam = len(spam_free) / total_spam
P_free_given_ham = len(ham_free) / total_ham

print("P(Word='free' | Spam):", P_free_given_spam)
print("P(Word='free' | Ham):", P_free_given_ham)

# 2. Comparison
print("\nComparison:")
if P_free_given_spam > P_free_given_ham:
    print("The word 'free' appears more frequently in Spam emails.")
else:
    print("The word 'free' appears more frequently in Ham emails.")

# 3. Interpretation
print("\nInterpretation:")
if P_free_given_spam > P_free_given_ham:
    print("'Free' is a strong indicator of Spam.")
else:
    print("'Free' is not a strong indicator of Spam.")


##########################################################################################


"""Q.1 Bayes' Theorem
Using only probability formulas and Python computations:
1. Calculate:
o Prior probabilities: P(Spam), P(Ham)
o Likelihoods: P("free"|Spam), P("free"|Ham)

2. Using Bayes' theorem, comput: P(Spam|"free")
3. Interpret the posterior probability. """

##########################################################################################

import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (9)\spam.csv", encoding='latin-1')

# Rename columns
df = df.rename(columns={'v1': 'Category', 'v2': 'Message'})

# Convert to lowercase
df['Message'] = df['Message'].str.lower()

# Total emails
```

```python
total = len(df)

# -------------------------
# PRIOR PROBABILITIES
# -------------------------
total_spam = (df['Category'] == 'spam').sum()
total_ham = (df['Category'] == 'ham').sum()

P_spam = total_spam / total
P_ham = total_ham / total

print("P(Spam):", P_spam)
print("P(Ham):", P_ham)

# -------------------------
# 2 LIKELIHOODS
# -------------------------
spam_free = df[(df['Category'] == 'spam') &
               (df['Message'].str.contains("free"))]

ham_free = df[(df['Category'] == 'ham') &
              (df['Message'].str.contains("free"))]

P_free_given_spam = len(spam_free) / total_spam
P_free_given_ham = len(ham_free) / total_ham

print("\nP('free' | Spam):", P_free_given_spam)
print("P('free' | Ham):", P_free_given_ham)

# -------------------------
# 3 BAYES THEOREM
# -------------------------
# P(Spam | free)
P_free = (P_free_given_spam * P_spam) + \
         (P_free_given_ham * P_ham)

P_spam_given_free = (P_free_given_spam * P_spam) / P_free

print("\nP(Spam | 'free'):", P_spam_given_free)

# -------------------------
# 4 INTERPRETATION
# -------------------------
print("\nInterpretation:")
if P_spam_given_free > 0.5:
    print("If an email contains 'free', it is likely Spam.")
else:
    print("If an email contains 'free', it is not strongly Spam.")


"""Q.2 Conditional Independence Assumption
Naive Bayes assumes that features are conditionally independent given the class.
1. Compute:
o P("free"∩"click"|Spam)
o P("free"|Spam)×P("click"|Spam)
2. Compare the values.
3. Comment on whether the conditional independence assumption holds for this dataset."""
import pandas as pd
```

```python
# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\archive (9)\spam.csv", encoding='latin-1')

# Rename columns
df = df.rename(columns={'v1': 'Category', 'v2': 'Message'})

# Convert to lowercase
df['Message'] = df['Message'].str.lower()

# Filter spam emails
spam_df = df[df['Category'] == 'spam']
total_spam = len(spam_df)

# --------------------------------
# 1 Compute Probabilities
# --------------------------------

# P("free" | Spam)
free_spam = spam_df[spam_df['Message'].str.contains("free")]
P_free_given_spam = len(free_spam) / total_spam

# P("click" | Spam)
click_spam = spam_df[spam_df['Message'].str.contains("click")]
P_click_given_spam = len(click_spam) / total_spam

# P("free" ∩ "click" | Spam)
free_click_spam = spam_df[
    spam_df['Message'].str.contains("free") &
    spam_df['Message'].str.contains("click")
]
P_free_and_click_given_spam = len(free_click_spam) / total_spam

# Product of probabilities
product_prob = P_free_given_spam * P_click_given_spam

print("P('free' ∩ 'click' | Spam):", P_free_and_click_given_spam)
print("P('free' | Spam) × P('click' | Spam):", product_prob)

# --------------------------------
# 2 Comparison
# --------------------------------
print("\nComparison:")
difference = abs(P_free_and_click_given_spam - product_prob)
print("Difference:", difference)

# --------------------------------
# 3 Interpretation
# --------------------------------
print("\nInterpretation:")
if difference < 0.01:
    print("Conditional independence approximately holds.")
else:
    print("Conditional independence does NOT strictly hold.")
```

```python
"""
Q.1 Data Preprocessing and Cleaning
Load the Titanic dataset and perform the following:
• Display summary statistics of the dataset.
• Handle missing values:
o Replace missing Age with median.
o Replace missing Embarked with mode.
o Drop the Cabin column.
• Convert categorical variables (Sex, Embarked) into numerical format.
• Drop irrelevant columns (Name, Ticket, PassengerId).
Display the cleaned dataset shape."""

import pandas as pd

# Load dataset
df = pd.read_csv("titanic.csv")   # Change path if needed

# 1 Display Summary Statistics
print("Summary Statistics:\n")
print(df.describe())

print("\nDataset Info:\n")
print(df.info())

# 2 Handle Missing Values

# Replace missing Age with median
df['Age'].fillna(df['Age'].median(), inplace=True)

# Replace missing Embarked with mode
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Drop Cabin column
df.drop(columns=['Cabin'], inplace=True)

# 3 Convert Categorical Variables

# Convert Sex to numeric
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Convert Embarked using one-hot encoding
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

# 4 Drop Irrelevant Columns
df.drop(columns=['Name', 'Ticket', 'PassengerId'], inplace=True)

# 5 Display Cleaned Dataset Shape
print("\nCleaned Dataset Shape:", df.shape)


"""Q.2 Exploratory Data Analysis
Perform EDA and answer the following using Python:
• Calculate survival rate overall.
• Calculate survival rate based on:
o Gender
o Passenger Class (Pclass)
• Plot:
o Count plot of survival.
```

```
o Bar chart of survival vs gender.
Interpret the results briefly in comments. """

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset

df = pd.read_csv(r"C:\Users\admin\Downloads\titanic\train.csv")   # Change path if needed

# ----------------------------
# 1 Overall Survival Rate
# ----------------------------
overall_survival = df['Survived'].mean()
print("Overall Survival Rate:", overall_survival)

# ----------------------------
# 2 Survival Rate by Gender
# ----------------------------
survival_gender = df.groupby('Sex')['Survived'].mean()
print("\nSurvival Rate by Gender:\n", survival_gender)

# ----------------------------
# 3 Survival Rate by Pclass
# ----------------------------
survival_class = df.groupby('Pclass')['Survived'].mean()
print("\nSurvival Rate by Passenger Class:\n", survival_class)

# ----------------------------
# 4 Count Plot of Survival
# ----------------------------
plt.figure()
sns.countplot(x='Survived', data=df)
plt.title("Count Plot of Survival")
plt.show()

# ----------------------------
# 5 Bar Chart: Survival vs Gender
# ----------------------------
plt.figure()
sns.barplot(x='Sex', y='Survived', data=df)
plt.title("Survival Rate by Gender")
plt.show()

# ----------------------------
# Interpretation (in comments)
# ----------------------------

# Interpretation:
# 1. Overall survival rate is around 38% (approx in standard dataset).
# 2. Females have significantly higher survival rate than males.
# 3. Passengers in 1st class have higher survival compared to 2nd and 3rd class.
# 4. Survival was influenced by gender and passenger class.


"""
Q.3. Logistic Regression Model Implementation
```

```
Using the cleaned dataset:
• Separate features (X) and target variable (Survived).
• Split data into 80% training and 20% testing.
• Train a Logistic Regression model.
• Predict survival on test data.
• Print model accuracy."""

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv(r"C:\Users\admin\Downloads\titanic\train.csv")

# Fill missing values (NO inplace)
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# Drop Cabin
df = df.drop(columns=['Cabin'])

# Convert categorical
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

# Drop irrelevant columns
df = df.drop(columns=['Name', 'Ticket', 'PassengerId'])

# Separate features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)



"""
Q.4 Model Evaluation
Evaluate the trained model using:
• Confusion Matrix
• Precision
• Recall
• F1-score
```

```
Explain in comments:
• What type of errors are more (False Positive or False Negative)?
• Which metric is more important in this problem and why?"""

from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score,
classification_report

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Precision
precision = precision_score(y_test, y_pred)
print("\nPrecision:", precision)

# Recall
recall = recall_score(y_test, y_pred)
print("Recall:", recall)

# F1 Score
f1 = f1_score(y_test, y_pred)
print("F1 Score:", f1)

# Optional: Full classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# ----------------------------------
# Interpretation (Printed in Output)
# ----------------------------------

TN, FP, FN, TP = cm.ravel()

print("\nError Analysis:")

if FP > FN:
    print("More False Positives than False Negatives.")
else:
    print("More False Negatives than False Positives.")

print("\nMetric Importance:")

print("In Titanic survival prediction, Recall is very important.")
print("Because missing a real survivor (False Negative) is more critical than predicting survival
incorrectly.")
print("F1-score is also useful as it balances Precision and Recall.")


"""Q.5 Feature Importance and ROC Analysis
• Display logistic regression coefficients.
• Identify top 3 features influencing survival.
• Plot ROC Curve.
• Calculate AUC score.
• Interpret whether the model is performing well or not."""

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score
```

```python
# ----------------------------
# 1 Display Logistic Regression Coefficients
# ----------------------------
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
})

print("Logistic Regression Coefficients:\n")
print(coefficients)

# ----------------------------
# 2 Top 3 Important Features
# ----------------------------
coefficients["Absolute"] = np.abs(coefficients["Coefficient"])
top3 = coefficients.sort_values(by="Absolute", ascending=False).head(3)

print("\nTop 3 Features Influencing Survival:\n")
print(top3[["Feature", "Coefficient"]])

# ----------------------------
# 3 ROC Curve
# ----------------------------
y_prob = model.predict_proba(X_test)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure()
plt.plot(fpr, tpr, label="ROC Curve")
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

# ----------------------------
# 4 AUC Score
# ----------------------------
auc_score = roc_auc_score(y_test, y_prob)
print("\nAUC Score:", auc_score)

# ----------------------------
# 5 Interpretation
# ----------------------------
if auc_score > 0.8:
    print("Model performance is very good.")
elif auc_score > 0.7:
    print("Model performance is good.")
else:
    print("Model performance is moderate or needs improvement.")
```